

Transaction Processing Pipeline

Created by Yevhen Kulinichenko

2026-07-06

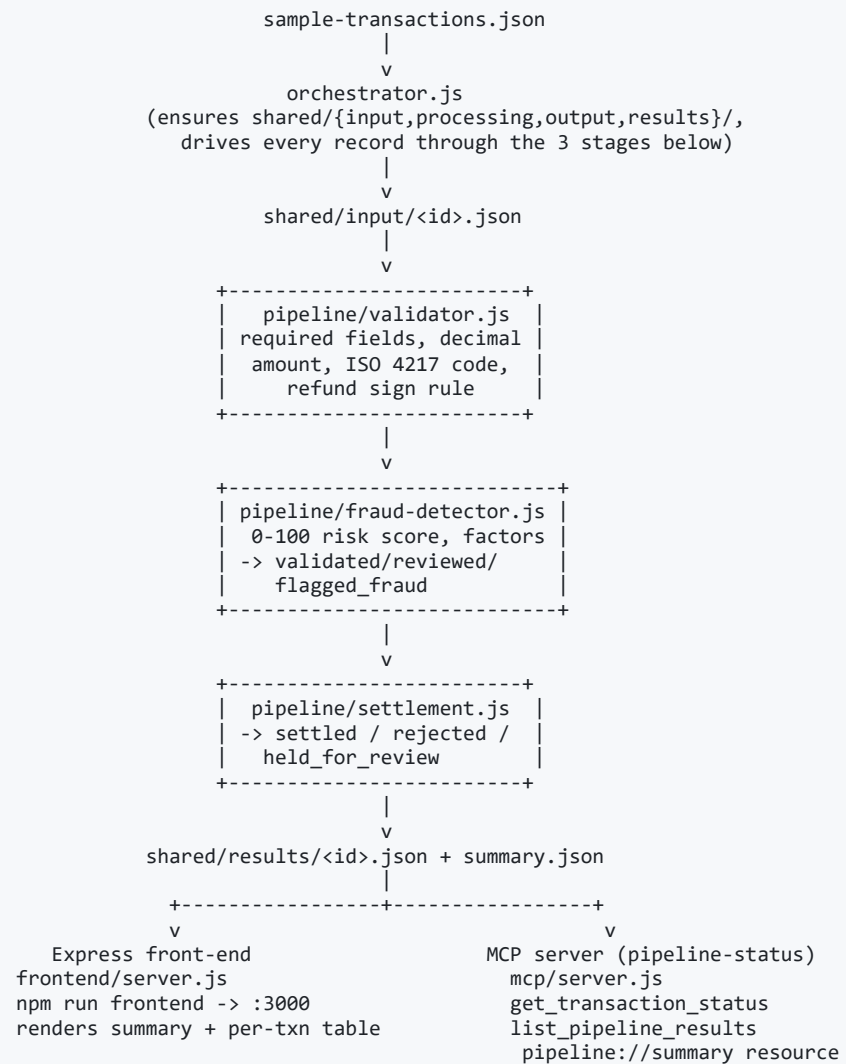
Capstone project — a file-based, three-stage Node.js transaction pipeline
(validation → fraud detection → settlement)

Problem / Objective

Build a Node.js/Express-compatible, file-based transaction processing pipeline that validates, fraud-scores, and settles the transactions in `sample-transactions.json` in sequence (validator → fraud detector → settlement), orchestrated end-to-end and reported on in `shared/results/`.

— `specification.md`, High-Level Objective

Architecture



Stage 1: Validator (`pipeline/validator.js`)

Checks required fields, parses `amount` with `decimal.js` (never `Number` / `parseFloat`), validates currency against an ISO 4217 allow-list (`USD`, `EUR`, `GBP`, `JPY`), and enforces the refund sign rule.

Example — TXN006 (currency `"XYZ"`, \$200 transfer):
rejected with `reason: "invalid_currency"` — `XYZ` is not on the allow-list.

Example — TXN007 (`transaction_type: "refund"`, `amount: "-100.00"`):
normalized via `Decimal.abs()` to `"100.00"` and marked `validated` — refund is the only type allowed a negative source amount.

Stage 2: Fraud Detector (`pipeline/fraud-detector.js`)

Sums five weighted risk factors (0-100, capped) against every non-rejected transaction: `high_value` (amount > \$10k, +40), `very_high_value` (amount > \$50k, +20), `unusual_timing` (00:00-06:00 UTC, +20), `cross_border` (`metadata.country !== "US"`, +15), `wire_transfer` (+10).

Score $\geq 60 \rightarrow$ `flagged_fraud`; $> 0 \rightarrow$ `reviewed`; else `validated`.

Example — TXN005 (\$75,000 `wire_transfer`, US, 10:00 UTC):

`high_value` (+40) + `very_high_value` (+20) + `wire_transfer` (+10) = **70**
 $\rightarrow$ `flagged_fraud`.

(Design note in the source: TXN002 scores exactly 50 by the same literal formula — `reviewed`, not flagged — resolving an ambiguity between the spec's prose and its own worked arithmetic in favor of the arithmetic.)

Stage 3: Settlement (`pipeline/settlement.js`)

Turns the fraud detector's `status` into a `final_status` :

`rejected` stays `rejected` (reason preserved); `flagged_fraud` becomes `held_for_review` (never auto-settled); anything else (`validated` / `reviewed`) becomes `settled` with a `settled_at` ISO 8601 timestamp.

Example — TXN001 (\$1,500 USD transfer, US, business hours):

scores 0 in fraud detection (`validated`) → settlement stamps

`final_status: "settled"` and `settled_at` , written to `shared/results/TXN001.json` .

The Four-Agent Workflow

#	Agent	Role
1	spec-writer	Writes <code>specification.md</code> before any code
2	pipeline-codegen	Builds <code>orchestrator.js</code> , <code>pipeline/*.js</code> , <code>frontend/</code> ; uses context7
3	unit-test-writer	Jest suite + coverage gate (80% threshold)
4	docs-writer	README, HOWTORUN, this deck

Defined in `.claude/agents/*.md`, each with a scoped `tools:` allow-list — no agent invents requirements outside `specification.md` / `TASKS.md`.

MCP Integration

context7 — used by `pipeline-codegen` before non-trivial library usage.

Real query from `research-notes.md` :

"Decimal constructor invalid string error handling, isNaN, abs(),
greaterThan comparison" → resolved to `/mikemcl/decimal.js` → confirmed
`new Decimal('abc')` *throws* rather than returning `NaN`, driving the
try/catch in `validator.js`'s `parseAmount()`.

Custom `pipeline-status` server (`mcp/server.js`, registered in
`mcp.json`, built on `@modelcontextprotocol/sdk`):

- Tool `get_transaction_status` — look up one transaction's final record
- Tool `list_pipeline_results` — aggregate counts across all results
- Resource `pipeline://summary` — raw `shared/results/summary.json`

Skills & Hooks

- `/run-pipeline` (`.claude/commands/run-pipeline.md`) — clears `shared/{input,processing,output,results}/` , runs `node orchestrator.js` , and renders `summary.json` plus rejected/held transactions as markdown tables.
- `/validate-transactions` (`.claude/commands/validate-transactions.md`) — read-only: runs `node pipeline/validator.js --dry-run` and tabulates valid/invalid counts and rejection reasons without touching `shared/` .
- **Coverage-gate hook** (`.claude/settings.json` `PreToolUse` on `Bash` → `scripts/check-coverage.js`) — intercepts any command containing `git push` , runs `jest --coverage` , and blocks the push if line coverage is below **80%**.

Demo

- Express front-end (`frontend/server.js` , `npm run frontend` → `:3000`): triggers a fresh `runPipeline()` and renders the summary + per-transaction table in-browser.

Screenshot pending: `docs/screenshots/frontend.png` (Step 011 — not yet captured; `docs/screenshots/` currently has no image files).

- CLI pipeline run (`npm run pipeline` / `node orchestrator.js`): prints a human-readable summary with counts and one line per rejected/held-for-review transaction.

Screenshot pending: `docs/screenshots/pipeline-run.png` (Step 011 — not yet captured).

Test Coverage

Measured with `npx jest --coverage` on this repo, right now:

Metric	Coverage
Statements	95.07% (193/203)
Branches	87.17% (68/78)
Functions	100% (18/18)
Lines	94.94% (188/198)

29 tests, 4 suites, all passing — well above the 80% coverage-gate threshold enforced on `git push`.

Lessons Learned

- `decimal.js` beats native `Number` for money math — but it *throws* on unparseable input instead of returning `NaN`; `context7` caught this before it crashed `validator.js`.
- **File-based message passing** (`input` → `processing` → `output` → `results`) makes every hand-off inspectable on disk, at the cost of extra I/O.
- **Spec ambiguity must be resolved in code, not silently** — the fraud detector documents why TXN002 (score 50) is `reviewed`, not flagged.
- **Scoped subagent tool allow-lists** (e.g. docs-writer can't touch `pipeline/`) kept each of the four agents honest about its own job.